

Build a Databricks Notebook-Based Unity Catalog Cross-Workspace Synchronization Utility

Role

Act as a **Senior Azure Databricks Architect, Unity Catalog Architect, PySpark/Python Engineer, and Databricks Migration Engineer.**

Build a production-grade **Unity Catalog synchronization/migration utility that runs entirely inside Azure Databricks as a Databricks Notebook and Databricks Job.**

CRITICAL ARCHITECTURAL CONSTRAINT

This is **NOT a standalone Python CLI utility.**

Do NOT build:

- VM-based application
- Docker application
- local Python CLI
- terminal-based utility
- external server application
- standalone executable

The utility must execute as:

```
Databricks Workspace
|
v
Databricks Notebook
|
v
Databricks Job
|
v
UC Sync Framework
|
+---- Databricks REST APIs
|
+---- Databricks SQL
|
+---- Unity Catalog
|
+---- Unity Catalog Volume
|
+---- Audit Delta Table
```

The notebook is the primary user interface and entry point.

The notebook must be capable of being scheduled/executed as a Databricks Workflow Job.

1. Objective

Create a reusable notebook-based utility to synchronize Unity Catalog metadata and definitions between:

```
SOURCE WORKSPACE / METASTORE
    |
    | inventory
    | export
    | transform
    | dependency resolution
    | import
    v
TARGET WORKSPACE / METASTORE
```

The utility is primarily intended for:

Azure Databricks region migration, specifically migration between the Central region and Jio region.

The utility should be designed so the same notebook/job can later be reused for other workspaces and BUs.

2. Notebook Entry Point

Create a primary notebook:

```
notebooks/UC_Sync_Main
```

The notebook should use Databricks widgets/parameters.

Example:

```
dbutils.widgets.text("mode", "INVENTORY")
dbutils.widgets.text("source_workspace_url", "")
dbutils.widgets.text("source_oauth_secret_scope", "")
dbutils.widgets.text("source_client_id_secret_key", "")
dbutils.widgets.text("source_client_secret_key", "")
```

```

dbutils.widgets.text("target_workspace_url", "")
dbutils.widgets.text("target_oauth_secret_scope", "")
dbutils.widgets.text("target_client_id_secret_key", "")
dbutils.widgets.text("target_client_secret_key", "")

dbutils.widgets.text("export_volume_path", "")

dbutils.widgets.text("catalogs", "")
dbutils.widgets.text("schemas", "")

dbutils.widgets.text("exclude_object_types", "")
dbutils.widgets.text("include_regex", "")
dbutils.widgets.text("exclude_regex", "")

dbutils.widgets.dropdown(
    "mode",
    "INVENTORY",
    [
        "INVENTORY",
        "EXPORT",
        "IMPORT",
        "SYNC",
        "COMPARE",
        "VALIDATE"
    ]
)

dbutils.widgets.dropdown(
    "dry_run",
    "true",
    ["true", "false"]
)

```

Do not hard-code credentials.

3. Job-Based Execution

The utility must be designed to run as a Databricks Job.

Provide an example Databricks Job configuration.

The Job should invoke:

```
UC_Sync_Main
```

with parameters.

Example conceptual job:

```
UC Migration Job
|
+-- Notebook Task
    |
    +-- UC_Sync_Main
        |
        +-- inventory
        +-- export
        +-- import
        +-- validation
```

Support Databricks Job parameters so the same notebook can run for different BUs.

Example:

```
mode = SYNC
catalogs = bu1_catalog,bu2_catalog
schemas = bu1_catalog.schema1
export_volume_path = /Volumes/migration/uc_exports/run_001
```

4. Authentication

The notebook runs inside Databricks.

For the SOURCE workspace, authenticate using OAuth/service principal credentials stored in:

- Databricks Secret Scope
- Azure Key Vault-backed secret scope where available

Example:

```
source_secret_scope
source_client_id_key
source_client_secret_key
```

The target workspace can use:

1. The current workspace's Databricks authentication context when the notebook itself is running in the target workspace.
2. Explicit OAuth credentials from a secret scope when operating against another workspace.

Never put credentials in:

- notebook source
- widgets
- exported manifests
- audit table
- logs
- DDL files

5. Execution Model

The notebook should behave as an orchestrator.

Do NOT put all implementation logic directly in notebook cells.

Use a Python package/module structure inside the Databricks workspace/repository.

Example:

```
repo/
|
+-- notebooks/
|   |
|   +-- UC_Sync_Main
|
+-- src/
|   |
|   +-- uc_sync/
|       |
|       +-- config.py
|       +-- auth.py
|       +-- workspace_client.py
|       +-- inventory.py
|       +-- export.py
|       +-- import_engine.py
|       +-- dependency.py
|       +-- audit.py
|       +-- validation.py
|       +-- mapping.py
|       +-- security.py
|       +-- models.py
|       +-- adapters/
```

The notebook should contain:

1. Parameter initialization
2. Configuration loading

3. Authentication initialization
4. Preflight validation
5. Mode selection
6. Call appropriate service
7. Display summary
8. Exit with success/failure status

All substantial logic belongs in `src/`.

6. Supported Modes

The notebook must support:

INVENTORY

```
Source Workspace
  |
  v
Inventory UC
  |
  v
Audit Table
```

EXPORT

```
Source Workspace
  |
  v
Inventory
  |
  v
Generate canonical manifest
  |
  v
Generate DDL
  |
  v
Write to UC Volume
  |
  v
Audit
```

IMPORT

```
Manifest in UC Volume
|
v
Dependency Planner
|
v
Target Workspace
|
v
Create UC objects
|
v
Audit
```

SYNC

```
Source
|
v
Inventory
|
v
Compare previous manifest/audit
|
v
Export changed objects
|
v
Dependency plan
|
v
Target
|
v
Import/reconcile
|
v
Validate
```

COMPARE

Compare source manifest with target.

VALIDATE

Validate source vs target after import.

7. Databricks Notebook UI

The notebook should display a clear execution summary.

At the beginning:

```
=====
Unity Catalog Synchronization Utility
=====

Run ID           : 20260815_001
Mode             : SYNC
Source Workspace : <source>
Target Workspace : <target>
Export Volume    : /Volumes/...
Catalogs        : finance,sales
Schemas        : finance.curated
Dry Run         : false

=====
```

At the end:

```
=====
Execution Summary
=====

Inventory
-----
Catalogs           : 12
Schemas           : 84
Tables             : 4,321
Views              : 823
Materialized Views : 61
Volumes            : 122

Export
-----
Successful         : 5,301
Failed             : 4
Excluded           : 187

Import
```



```

-----
Created                : 5,102
Updated                : 31
Skipped                : 168
Failed                 : 4

Validation
-----
MATCH                  : 5,001
DIFFERENT              : 99
MISSING                : 4
MANUAL ACTION          : 21

Overall Status         : COMPLETED_WITH_WARNINGS
=====

```

Use DataFrames where appropriate so users can inspect:

```

display(summary_df)
display(failed_objects_df)
display(differences_df)

```

8. Configuration

Support notebook widgets as the primary configuration mechanism.

Also support an optional YAML/JSON configuration stored in a UC Volume.

Example:

```

source:
  workspace_url: "https://..."
  secret_scope: "uc-migration"
  client_id_key: "source-client-id"
  client_secret_key: "source-client-secret"

target:
  workspace_url: "https://..."
  secret_scope: "uc-migration"
  client_id_key: "target-client-id"
  client_secret_key: "target-client-secret"

export:
  volume_path: "/Volumes/migration/uc_exports"

selection:

```

```
catalogs:
  - finance
  - sales

schemas:
  - finance.raw
  - finance.curated

exclude_object_types:
  - TABLE

include_regex:
  - "^finance.*"

exclude_regex:
  - ".*_TEMP$"
  - "^test_.*"
```

Widget values should override configuration-file defaults.

9. Unity Catalog Volume

The export package MUST be stored in a Unity Catalog Volume.

Example:

```
/Volumes/
  migration/
    uc_exports/
      run_20260815_001/
```

Do not use:

```
/tmp
/dbfs/tmp
local filesystem
VM filesystem
```

as the persistent export location.

The export package must survive cluster termination.

10. Export Package

Create:

```
run_<run_id>/
|
+-- manifest.json
|
+-- inventory/
|
+-- ddl/
|
+-- metadata/
|
+-- grants/
|
+-- bindings/
|
+-- validation/
|
+-- checksums/
|
+-- logs/
```

The package must be completely self-describing.

11. Inventory

Inventory all supported Unity Catalog objects.

At minimum:

Hierarchy

- Catalog
- Schema

Data objects

- Managed tables
- External tables
- Views
- Dynamic views
- Materialized views
- Streaming tables where applicable
- Managed volumes
- External volumes

Functions/Models

- Functions
- Registered models

Storage/Security

- Storage credentials
- Service credentials
- External locations

Federation

- Connections
- Foreign catalogs where applicable

Sharing

- Shares
- Recipients
- Providers

Governance

- Owners
- Grants
- Tags
- Row filters
- Column masks
- Workspace bindings
- Other supported UC securables

Create an extensible object adapter framework.

12. Source Last Modified Timestamp

For every object attempt to retrieve:

```
source_last_modified_at
```

from the SOURCE workspace REST API where the API exposes it.

Do NOT use inventory time as a substitute.

Store:

```
source_last_modified_at
source_last_modified_source
```

Example:

```
source_last_modified_source =
  REST_API
  SQL
  NOT_AVAILABLE
```

If Databricks does not expose a reliable source modification timestamp for a particular object type:

```
source_last_modified_at = NULL
```

Do not fabricate it.

13. Audit Table

Create ONE managed Delta audit table.

Example:

```
CREATE TABLE IF NOT EXISTS <audit_catalog>.<audit_schema>.uc_sync_audit
(
  run_id STRING,
  operation_id STRING,

  source_workspace_url STRING,
  target_workspace_url STRING,

  source_metastore_id STRING,
  target_metastore_id STRING,

  catalog_name STRING,
  schema_name STRING,
  object_name STRING,
  full_name STRING,
  object_type STRING,

  source_object_id STRING,

  source_created_at TIMESTAMP,
  source_last_modified_at TIMESTAMP,
  source_last_modified_source STRING,
```

```

inventory_created_at TIMESTAMP,
export_created_at TIMESTAMP,
import_created_at TIMESTAMP,
validation_created_at TIMESTAMP,

export_status STRING,
import_status STRING,
validation_status STRING,

source_definition_hash STRING,
exported_definition_hash STRING,
target_definition_hash STRING,

export_path STRING,
ddl_path STRING,

dependency_level INT,
import_order INT,

operation_mode STRING,

error_code STRING,
error_message STRING,

metadata_json STRING,

utility_version STRING,
created_at TIMESTAMP,
updated_at TIMESTAMP
)
USING DELTA;

```

The table must be managed in Unity Catalog.

14. Audit Lifecycle

Every object must have an auditable lifecycle:

```

DISCOVERED
|
v
INVENTORIED
|
v
EXPORTED
|
v
IMPORTED

```

```
|  
v  
VALIDATED
```

For example:

```
inventory_created_at  
export_created_at  
import_created_at  
validation_created_at
```

must be populated independently.

Do not overwrite these timestamps.

15. Object Filtering

Support:

```
Catalog list  
Schema list  
Object type exclusions  
Object include regex  
Object exclude regex
```

Examples:

```
catalogs:  
  finance,sales  
  
schemas:  
  finance.raw,finance.curated
```

Object exclusion:

```
TABLE  
VIEW  
MATERIALIZED_VIEW
```

Regex:

```
^tmp_.*  
.*_backup$  
.*_TEMP$
```

Apply filters before expensive metadata retrieval where possible.

16. Canonical Metadata Model

Do NOT make DDL the canonical representation.

Create a normalized object model:

```
UCObject
```

containing:

```
object_type  
catalog  
schema  
name  
full_name  
object_id  
owner  
created_at  
last_modified_at  
definition  
properties  
tags  
grants  
dependencies  
bindings  
source_metadata
```

DDL is generated from this model.

This is important because some UC objects cannot be fully represented by a single SQL DDL statement.

17. DDL Generation

Use:

1. Databricks REST API metadata
2. Databricks SDK

3. `SHOW CREATE`
4. `DESCRIBE`
5. `SHOW GRANTS`
6. Appropriate SQL metadata commands

Use the authoritative Databricks mechanism for each object type.

Do not manually reconstruct complex definitions unnecessarily.

18. Credentials

Credentials require special handling.

Never export:

```
client secrets
passwords
private keys
tokens
OAuth secrets
cloud credentials
```

Export metadata only.

For example:

```
credential_name
credential_type
owner
comment
grants
```

Generate a migration requirement:

```
MANUAL_CONFIGURATION_REQUIRED
```

Support target mappings:

```
storage_credentials:
  source_credential:
    target_credential: target_credential
```

19. External Locations

Capture:

```
name
URL
storage credential
owner
comment
read_only
properties
grants
workspace bindings
```

Support source-to-target mappings:

```
external_locations:
  source_location:
    target_name: target_location
    target_url: "abfss://..."
    target_storage_credential: target_credential
```

Do not blindly reuse source Central-region ADLS URLs in Jio.

20. Tables

Support:

- Managed tables
- External tables

Capture:

```
columns
types
nullability
comments
partitioning
clustering
liquid clustering metadata where applicable
table properties
generated columns
identity columns
constraints
tags
```

```
owner
grants
location
```

Do not migrate physical data as part of this utility.

Clearly distinguish:

```
UC METADATA MIGRATION
```

from:

```
PHYSICAL DATA MIGRATION
```

21. Views

Support:

- Standard views
- Dynamic views

Capture:

```
definition
columns
properties
owner
grants
tags
dependencies
```

Views must be created only after dependent objects exist.

22. Materialized Views

Capture:

```
definition
dependencies
properties
owner
grants
```

```
tags
refresh schedule
refresh configuration
trigger configuration where applicable
```

Do not automatically trigger an expensive refresh unless explicitly configured.

Support:

```
preserve_refresh_schedule = true
initial_refresh = false
```

23. Volumes

Support:

- Managed volumes
- External volumes

Capture:

```
name
volume type
storage location
owner
comment
grants
```

External volumes must resolve their external-location dependency first.

Managed volume storage must be mapped to the target region where required.

24. Functions

Support UC functions.

Capture:

```
name
signature
parameters
return type
language
```

```
body
comment
owner
properties
grants
dependencies
```

25. Models

Inventory UC registered models where supported.

Separate:

```
model metadata
model versions
model artifacts
permissions
```

If artifact migration is not implemented:

```
migration_status = METADATA_ONLY
```

26. Connections / Foreign Catalogs

Support where applicable.

Never export connection secrets.

Use mapping configuration for target connections.

27. Delta Sharing

Support:

```
Shares
Recipients
Providers
```

Export metadata and permissions.

Do not export tokens or secrets.

28. Grants

Grants must be a first-class migration artifact.

Capture:

```
principal
privilege
grantor
object
```

Support principal mapping:

```
principals:
  "source-group":
    "target-group"

  "source-service-principal":
    "target-service-principal"
```

Import grants only after the target object exists.

29. Workspace Bindings

Capture applicable workspace bindings.

Do not blindly copy source workspace IDs.

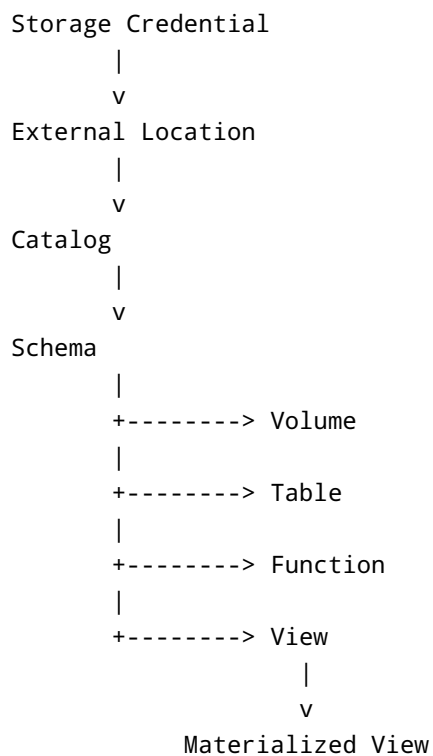
Provide:

```
workspaces:
  source_workspace_id:
    target_workspace_id
```

30. Dependency Graph

Build a dependency graph.

Example:



Perform topological sorting.

Store:

```
dependency_level
import_order
```

in the audit table.

31. Import Engine

The import engine executes from inside the Databricks Job.

Do NOT require:

```
external VM
external server
local terminal
```

The notebook cluster is the execution environment.

Use:

```
REST API
Databricks SDK
SQL Statement Execution
```

to interact with the target workspace.

32. Target Workspace Context

The notebook should support two deployment models.

Model A — Notebook runs in TARGET workspace

```
Target Databricks Workspace
|
+-- UC Sync Notebook
|
+-- Source Workspace API
|
+-- Target Workspace local APIs/SQL
```

This is the preferred deployment model for the migration.

Model B — Notebook runs in a dedicated migration workspace

```
Migration Workspace
|
+---- Source Workspace API
|
+---- Target Workspace API
```

Both should be supported, but no external compute is allowed.

33. Import Modes

Support:

```
CREATE_ONLY
CREATE_OR_SKIP
CREATE_OR_UPDATE
```



```
RECONCILE
DRY_RUN
```

Default:

```
DRY_RUN
```

for the first execution.

Never drop/recreate objects by default.

Require explicit:

```
allow_destructive_operations = true
```

for destructive changes.

34. Idempotency

Before creating:

```
Does target exist?
|
+-- NO --> CREATE
|
+-- YES
    |
    +-- Same hash --> SKIP
    |
    +-- Different --> RECONCILE
```

Use canonical SHA-256 hashes.

35. Incremental Sync

Support incremental processing.

Use:

```
source_last_modified_at
```

where reliable.

Also compare:

```
source_definition_hash
target_definition_hash
```

Never rely solely on timestamps.

36. Validation

After import compare:

```
SOURCE
vs
TARGET
```

Validate:

```
object existence
object type
definition
columns
properties
tags
owner
grants
location
dependencies
refresh configuration
workspace bindings
```

Return:

```
MATCH
DIFFERENT
MISSING_TARGET
EXTRA_TARGET
MANUAL_ACTION_REQUIRED
ERROR
```

Store validation results in the audit table.

37. Notebook Job Failure Semantics

The notebook must fail the Databricks Job only when appropriate.

For example:

```
All objects successful
  → SUCCESS

Some objects failed but execution completed
  → COMPLETED_WITH_WARNINGS

Critical preflight failure
  → FAILED

Authentication failure
  → FAILED

Target permission failure
  → FAILED
```

Use:

```
dbutils.notebook.exit(...)
```

with structured JSON status.

For fatal failures, raise an exception so the Databricks Job is marked failed.

38. Job Parameters for BU-Based Migration

The utility must support parameterization for your larger migration program.

Example:

```
BU = BU001

catalogs = bu001_catalog
schemas = bu001_catalog.raw,bu001_catalog.curated

source_workspace_url = ...
target_workspace_url = ...
```

```
export_volume_path =  
/Volumes/migration/uc_exports/BU001/run_001
```

The same notebook should be reusable across multiple BUs without code changes.

39. Audit Table Must Support Multiple Runs

Example:

```
BU001  
|  
+-- Run 001  
+-- Run 002  
+-- Run 003  
  
BU002  
|  
+-- Run 001  
+-- Run 002
```

Every run must have:

```
run_id
```

Every object operation must have:

```
operation_id
```

Do not delete previous audit history.

40. Security

Use Databricks-native security.

Preferred:

```
Secret Scope  
|  
v  
Notebook  
|  
v
```

```
OAuth
  |
  v
Source/Target Workspace
```

Do not use:

```
.env
hard-coded secrets
local config files containing secrets
```

Never print tokens.

Redact secrets from exception messages.

41. Required Source Permissions

Create documentation identifying required permissions for:

```
Source workspace:
- UC metadata inventory
- SHOW commands
- REST API access
- grants visibility
- object metadata access

Target workspace:
- catalog creation
- schema creation
- table/view creation
- volume creation
- external location access
- credential management where applicable
- grant management
```

Do not assume workspace admin automatically solves every UC permission requirement.

Perform a preflight check.

42. Preflight Notebook Cell

The first execution stage should be:

PRE-FLIGHT

Source Authentication	PASS
Source Workspace	PASS
Source UC Access	PASS
Source Inventory Access	PASS
Target Authentication	PASS
Target Workspace	PASS
Target UC Access	PASS
Target Create Permissions	PASS
Target Grant Permissions	PASS
Export Volume	PASS
Credential Mapping	WARNING
External Location Mapping	WARNING
Overall	READY

If a critical prerequisite fails, stop before modifying anything.

43. Cluster Requirements

The utility must document its recommended Databricks Job cluster configuration.

Do NOT design around a VM outside Databricks.

The utility should run on a Databricks Job cluster or serverless compute where supported.

Use a small/medium driver unless large-scale parallel API operations require more.

Do not assume Spark executors are required for every operation.

Prefer Python/REST/API operations for metadata.

Use Spark only where it provides a meaningful advantage.

44. Parallelism

Inventory operations may be parallelized carefully.

Example:

```

Catalog discovery
  |
  v
Schemas
  |
  +----- parallel object inventory
          |
          +-- Tables
          +-- Views
          +-- Volumes
          +-- Functions

```

But respect:

- Databricks API rate limits
- SQL warehouse limits
- UC concurrency
- dependency ordering

Make concurrency configurable:

```

max_api_workers
max_sql_concurrency

```

45. No External Runtime Dependency

The solution must not require:

```

VM
Kubernetes
Docker
Azure Function
Azure Container App
local machine
external scheduler

```

The only execution environment required should be:

```

Azure Databricks
Databricks Notebook
Databricks Job
Unity Catalog
Unity Catalog Volume

```

External Azure services may be used only for authentication/secrets where required.

46. Testing

Tests must run from the Databricks development environment where appropriate.

Provide:

```
unit tests
mock API tests
manifest tests
DDL generation tests
dependency tests
audit tests
filter tests
security tests
```

Do not require an external VM to execute tests.

47. Deliverables

Create:

```
notebooks/
  UC_Sync_Main

src/
  uc_sync/
    auth.py
    config.py
    workspace_client.py
    inventory.py
    export.py
    import_engine.py
    dependency.py
    audit.py
    validation.py
    mapping.py
    models.py
    security.py
    adapters/

tests/

docs/
```



```
architecture.md
configuration.md
permissions.md
uc-object-support-matrix.md
api-mapping.md
dependency-model.md
job-deployment.md
troubleshooting.md

configs/
  example.yaml
```

The notebook is the only user-facing execution entry point.

48. Databricks Job Definition

Provide an example Databricks Asset Bundle / Job configuration if the repository uses Databricks Asset Bundles.

Example conceptual structure:

```
resources:
  jobs:
    uc_sync_job:
      name: "UC-Sync"

      tasks:
        - task_key: "uc_sync"

      notebook_task:
        notebook_path: "../notebooks/UC_Sync_Main"

      base_parameters:
        mode: "SYNC"
        catalogs: ""
        schemas: ""
        exclude_object_types: ""
        include_regex: ""
        exclude_regex: ""
        dry_run: "true"
```

The exact syntax must match the current Databricks Asset Bundle specification.

49. Important Design Rule

The notebook should be thin.

Bad:

```
UC_Sync_Main
|
+-- 3,000 lines of Python
```

Good:

```
UC_Sync_Main
|
+-- configuration
+-- orchestration
+-- display
    |
    v
    src/uc_sync/
```

The notebook is the UI/orchestrator.

The Python package contains the implementation.

50. Implementation Sequence

Before implementing:

1. Inspect the repository.
2. Identify whether Databricks Asset Bundles are already used.
3. Identify existing notebooks and Python modules.
4. Create architecture documentation.
5. Create UC object support matrix.
6. Map each UC object to:
7. REST API
8. SDK
9. SQL
10. unsupported/manual
11. Identify source permissions.
12. Identify target permissions.
13. Identify which object types expose source last-modified timestamps.

14. Design audit schema.
15. Design canonical manifest.
16. Design dependency graph.
17. Design target mapping.
18. Only then implement.

51. Required Final Notebook Behavior

When the notebook starts:

```
[1/7] Loading configuration
[2/7] Authenticating
[3/7] Running preflight
[4/7] Inventorying Unity Catalog
[5/7] Exporting metadata
[6/7] Importing to target
[7/7] Validating
```

Display progress.

At completion:

```
=====
UC SYNC COMPLETED
=====

Run ID: 20260815_001

Objects inventoried : 10,245
Objects exported    : 10,201
Objects imported    : 10,176
Objects validated   : 10,176

Success            : 10,102
Warnings           : 51
Failures           : 23

Audit table:
<catalog>.<schema>.uc_sync_audit

Export package:
<volume path>

Overall status:
COMPLETED_WITH_WARNINGS
=====
```

52. Final Engineering Constraints

The implementation **MUST** satisfy all of the following:

- Databricks Notebook is the entry point.
- Databricks Job executes the notebook.
- No VM execution.
- No terminal execution.
- No standalone CLI.
- No external application server.
- No hard-coded credentials.
- OAuth/service principal authentication.
- Secrets retrieved from Databricks Secret Scope/approved secret mechanism.
- Export artifacts stored in Unity Catalog Volume.
- One managed Delta audit table.
- Source last-modified timestamp retrieved from source REST API where available.
- Inventory timestamp recorded.
- Export timestamp recorded.
- Import timestamp recorded.
- Validation timestamp recorded.
- Catalog/schema filtering.
- Object-type filtering.
- Regex include/exclude filtering.
- Dependency-aware import.
- Idempotent import.
- Dry-run mode.
- Incremental synchronization.
- Credential mapping.
- External-location mapping.
- Principal mapping.
- Workspace mapping.
- Managed-storage mapping.
- Grants migration.
- Materialized-view refresh configuration.
- Volume migration metadata.
- Unsupported-object reporting.
- Source/target comparison.
- Validation reporting.
- Restartable execution.
- Retry handling.
- API pagination.
- API throttling protection.
- Secret redaction.
- Comprehensive audit trail.
- Job-friendly exit status.

Build this as a **Databricks-native migration framework**, not as an external Python application.